Análisis de particiones fijas: First Fit vs Best Fit

Tarea Programada 3 · Simulador de memoria · Sistemas Operativos IIIC 2026 · LEAD University · Estudiante: Daniel Vásquez González

1. Estructuras de datos

- **Partición** (`Particion`): `id` (entero único), `tamano` (KB, fijo), `proceso_id` (`None` si está libre) y `usado` (KB del proceso que la ocupa). De ahí salen dos valores derivados: `libre` y `sobrante = tamano − usado`, que es fragmentación interna si está ocupada y capacidad disponible si está libre. Las particiones se guardan en una **lista** en el orden en que se definieron, porque First Fit depende de ese orden.
- **Proceso** (`Proceso`): `id`, `tamano`, `estado` (Esperando, En memoria o Finalizado) y `particion` asignada.
- **Procesos activos**: un **diccionario** `id → Proceso` (esperando o en memoria). Permite validar en tiempo constante que no haya dos procesos activos con el mismo id y localizar el proceso que se quiere finalizar. Los procesos terminados pasan a una lista aparte.
- **Cola de espera**: una `deque` con los ids de los procesos pendientes en orden de llegada. Se agrega al final y se recorre desde el principio cada vez que se libera una partición.
- Todo esto vive en la clase `Simulador`, que además valida (tamaños > 0, ids únicos, no cambiar particiones ni estrategia con procesos en el sistema) y calcula las estadísticas. El menú solo lee datos e imprime resultados.

2. Implementación de First Fit y Best Fit

Cada algoritmo es una función independiente que recibe la lista de particiones y el tamaño del proceso y devuelve la partición elegida (o `None` si ninguna sirve); un diccionario asocia la estrategia seleccionada con su función. Ninguna usa bibliotecas que resuelvan el problema.

- **First Fit** (`buscar_first_fit`): recorre las particiones en el orden de definición y devuelve la **primera libre con `tamano ≥ proceso`**. En el peor caso revisa todas, pero normalmente se detiene antes.
- **Best Fit** (`buscar_best_fit`): recorre **todas** las particiones libres suficientes y se queda con la de menor par `(tamano, id)`. Así elige la más ajustada y, ante un empate de tamaño, la de menor identificador.
- En ambas, el proceso ocupa la partición completa: el sobrante no se reasigna. Cuando ninguna partición libre sirve, el proceso pasa a la cola. Al finalizar un proceso se libera su partición y se recorre **toda** la cola en orden de llegada: los que no caben siguen esperando, pero se siguen revisando los demás para aprovechar otras particiones libres.
- Un proceso mayor que la partición más grande se rechaza (sección 3.2 del enunciado), porque nunca podría asignarse.

3. Evidencias de las pruebas

Cada prueba se ejecuta con `python particiones.py < casos/<archivo>.txt` y su salida completa está en `evidencias/`. Configuración base: particiones de 120, 300, 200 y 500 KB (1120 KB) con First Fit, salvo que se indique otra.

Tabla 1. Pruebas obligatorias

Esc.	Situación	Datos	Resultado observado
1	Cabe exactamente en una partición libre	P1 = 120 KB	Partición 1; fragmentación 0 KB
2	Cabe en varias particiones de distinto tamaño	P1 = 180 KB	First Fit: partición 2 (300 KB, sobran 120). Best Fit: partición 3 (200 KB, sobran 20)
3	No cabe en ninguna partición libre	P1 = 500 KB, luego P2 = 400 KB	P1 ocupa la 4; P2 queda en la cola
4	Requiere más que cualquier partición	P1 = 600 KB	Rechazado por imposibilidad de asignación; no entra a la cola
5	Todas las particiones ocupadas	Particiones 100 y 200; P1 = 90, P2 = 150, P3 = 50	P3 espera aunque su tamaño es pequeño: no hay particiones libres
6	Finalizar permite asignar uno en espera	Particiones 100 y 200; P1 = 90, P2 = 150, P3 = 180; termina P2	P3 sale de la cola y entra a la partición 2; la cola queda vacía
7	Memoria libre suficiente, pero ninguna partición alcanza	Particiones 100, 100 y 300; P1 = 250, P2 = 200	Quedan 200 KB libres (100 + 100), pero P2 espera; el programa lo avisa
8	First y Best Fit asignan distinto	P1 = 180, P2 = 110, P3 = 250	Ver sección 4

Además, `errores_validacion` comprueba que se rechazan tamaños ≤ 0 o no numéricos, identificadores de partición o de proceso repetidos o vacíos, finalizar un proceso inexistente, ya terminado o en la cola, y cambiar la estrategia o las particiones con procesos en el sistema. En todos los casos el programa muestra el error y continúa.

Escenario 7 (extracto de `evidencias/escenario_7_memoria_libre_suficiente.out.txt`):

```
No hay una partición libre donde quepan 200 KB:
P2 pasa a la cola de espera.
Hay 200 KB libres en total, pero ninguna
partición libre alcanza por sí sola.
Partición  Tamaño  Estado   Proceso  No utilizada
        1  100 KB  Libre    —              100 KB
        2  100 KB  Libre    —              100 KB
        3  300 KB  Ocupada  P1              50 KB
En espera: P2 (200 KB)
```

Escenario 6 (extracto de `evidencias/escenario_6_finalizar_asigna_espera.out.txt`):

```
P2 finalizó y la partición 2 (200 KB) quedó libre.
P3 (180 KB) sale de la cola y entra a la
partición 2 (200 KB). Fragmentación interna: 20 KB.
Partición  Tamaño  Estado   Proceso  No utilizada
        1  100 KB  Ocupada  P1              10 KB
        2  200 KB  Ocupada  P3              20 KB
La cola de espera está vacía.
```

4. Comparación de First Fit y Best Fit

Tabla 2. Escenario 8 (ejemplo del enunciado): P1 = 180 KB, P2 = 110 KB, P3 = 250 KB

Proceso	First Fit		Best Fit	
	Partición	**Fragmentación**	**Partición**	**Fragmentación**
P1 (180)	2 (300 KB)	120 KB	3 (200 KB)	20 KB
P2 (110)	1 (120 KB)	10 KB	1 (120 KB)	10 KB
P3 (250)	4 (500 KB)	250 KB	2 (300 KB)	50 KB
Fragmentación interna total	**380 KB**		**80 KB**	
Memoria libre (partición libre)	200 KB (partición 3)		500 KB (partición 4)	
Utilización efectiva	48,21 % (540 de 1120 KB)		48,21 % (540 de 1120 KB)	

En este escenario Best Fit reduce la fragmentación interna de 380 KB a 80 KB y, además, deja libre la partición más grande (500 KB). La utilización efectiva es idéntica porque ambas estrategias ubicaron los mismos procesos (540 KB): **lo que cambia es cuánto se desperdicia dentro de las particiones, y qué particiones quedan libres para futuros procesos**. Las cifras de ambas corridas cumplen *total = utilizada + fragmentación + libre* (1120 = 540 + 380 + 200 y 1120 = 540 + 80 + 500).

Tabla 3. Caso adicional: particiones 500, 200 y 150 KB; llegan P1 = 180, P2 = 120, P3 = 140 KB y luego termina P2

Momento	First Fit	Best Fit
P1 (180 KB)	Partición 1 (500): frag. 320	Partición 2 (200): frag. 20
P2 (120 KB)	Partición 2 (200): frag. 80	Partición 3 (150): frag. 30
P3 (140 KB)	Partición 3 (150): frag. 10	Partición 1 (500): frag. 360
Fragmentación total antes de terminar P2	410 KB	410 KB
Después de terminar P2 (quedan P1 y P3)	**330 KB**	**380 KB**

5. Preguntas de análisis

5.1 ¿Por qué First Fit y Best Fit pueden aplicarse a particiones fijas aunque normalmente se estudien con particiones variables?

Ambos algoritmos resuelven una sola pregunta: dado un conjunto de espacios libres candidatos, ¿cuál se elige para un proceso? En particiones variables los candidatos son huecos que aparecen y se dividen dinámicamente; en particiones fijas los candidatos son las particiones libres que ya existen. La regla de selección (la primera que sirva o la más ajustada) es la misma. La única diferencia es lo que ocurre después de elegir: en variables el hueco se divide y el resto sigue disponible, mientras que en fijas la partición se consume completa. Por eso aquí las estrategias solo cambian *cuál* partición se desperdicia, no *si* se desperdicia.

5.2 ¿Por qué el espacio sobrante dentro de una partición ocupada no puede utilizarse para otro proceso?

Porque la partición es la unidad mínima de asignación: el sistema solo sabe que está libre u ocupada por un proceso, y sus límites están definidos de antemano. Aprovechar el sobrante exigiría dividir la partición en una nueva (lo que el enunciado prohíbe) o llevar el control de huecos dentro de ella, es decir, pasar a un esquema de particiones variables con su tabla de espacios y su mayor complejidad. Además, la protección entre procesos se basa en que cada partición pertenece a uno solo. Por eso el sobrante es fragmentación interna: memoria que existe pero queda atrapada dentro de un bloque ya asignado.

5.3 ¿En qué circunstancias Best Fit puede producir una mayor fragmentación interna total que First Fit?

Cuando la elección ajustada para un proceso temprano obliga a un proceso posterior a usar una partición muy grande. La Tabla 3 lo muestra: Best Fit coloca P1 y P2 en las particiones pequeñas (200 y 150 KB) y deja la de 500 KB para P3, que solo necesita 140 KB. First Fit hizo lo contrario: P1 cae en la de 500 KB porque es la primera que sirve, y las pequeñas quedan para los procesos siguientes. Mientras los tres procesos están en memoria la fragmentación es igual (410 KB), pero al terminar P2 First Fit queda en 330 KB y Best Fit en 380 KB, porque el proceso pequeño de Best Fit sigue ocupando la partición de 500 KB. En general, Best Fit decide mirando solo el proceso actual y no sabe qué procesos llegarán ni cuáles terminarán primero, así que su ventaja depende del orden de llegada y de finalización, no está garantizada.

5.4 ¿Es posible tener suficiente memoria libre total y aun así no poder asignar un proceso?

Sí. Es el escenario 7: con particiones de 100, 100 y 300 KB, después de ubicar P1 (250 KB) en la de 300 KB, quedan 200 KB libres en total, pero repartidos en dos particiones de 100 KB. P2 necesita 200 KB y debe ir en una sola partición, por lo que espera en

la cola aunque la suma alcanza. La memoria libre está fragmentada entre particiones, un efecto análogo a la fragmentación externa: lo que importa no es cuánta memoria hay libre sino si algún bloque contiguo y asignable es suficientemente grande. Y como no hay compactación ni división de particiones, no hay forma de juntar esos 200 KB.

5.5 ¿Cuál sería el impacto de permitir que los procesos se distribuyan entre varias particiones? ¿Qué cambios implicaría?

- **Beneficios:** el escenario 7 desaparece (P2 usaría las dos particiones de 100 KB), se aprovecha mejor la memoria libre y ya no habría que rechazar procesos mayores que la partición más grande, siempre que la suma alcance. La fragmentación interna se limitaría al último fragmento de cada proceso.
- **Cambios en el modelo:** un proceso ya no tendría *una* partición sino una *lista* de ellas, y cada partición dejaría de ser sinónimo de proceso. Habría que guardar por proceso una tabla con sus fragmentos, liberar varias particiones al terminar y decidir cuántas particiones usar y en qué orden (los algoritmos First Fit y Best Fit pasarían a elegir un conjunto de particiones, no una).
- **Traducción de direcciones:** el proceso ve un espacio de direcciones contiguo, pero la memoria física asignada no lo es. Un solo par base/límite ya no basta; se necesita una tabla que traduzca cada dirección lógica a su partición y desplazamiento. Es, en esencia, la idea de la **paginación** (y la segmentación), con el costo extra de la traducción en cada acceso a memoria.
- **Costos:** más metadatos, más complejidad en la protección entre procesos y la necesidad de definir un tamaño de fragmento (que en paginación se resuelve con marcos de tamaño uniforme).

Reproducción: `python particiones.py < casos/escenario_8_distinta_asignacion_first.txt` (y `_best`); el resto de las salidas están en la carpeta `evidencias/`.